

# ЛЕКЦИЯ 8

JAVASCRIPT. ЧАСТЬ 1

# ВОПРОСЫ

1. Введение в JavaScript
2. Переменные
3. Типы данных
4. Диалоговые окна в JavaScript
5. Преобразование типов
6. Базовые математические операции
7. Операторы сравнения
8. Условное ветвление
9. Логические операторы
10. Циклы while и for
11. Функции

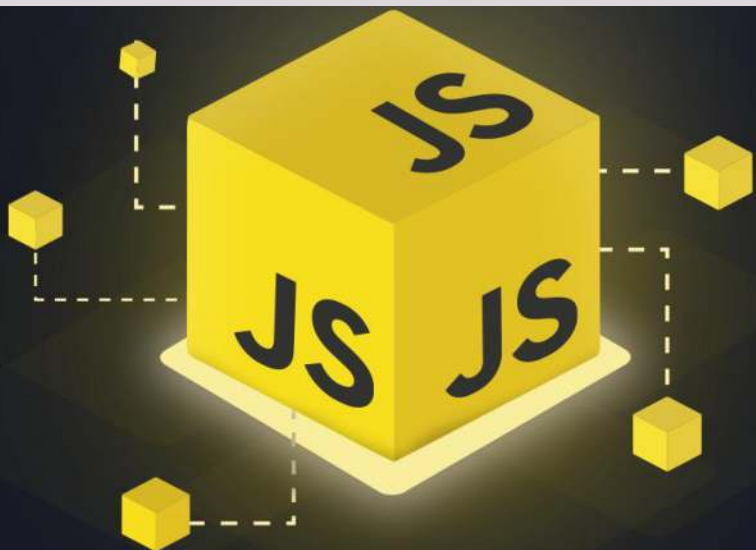
1

# ВВЕДЕНИЕ В JAVASCRIPT

# Для чего нужен JavaScript?

**JavaScript** — это язык программирования, который в первую очередь применяют в вебе.

С его помощью сайты делают интерактивными. Его называют главным языком фронтенда.



- Сделать сайт интерактивным, чтобы это была динамичная программа, реагирующая на действие пользователей. Например, если нажать кнопку «Нравится» и число лайков на записи увеличивается сразу, без обновления страницы.
- Передавать информацию между пользователем и сервером. Именно скрипты, написанные на JavaScript, отправляют на сервер информацию, которую ввёл пользователь.
- Анимировать объекты – добавить скролл до раздела по нажатию кнопки, анимацию фона, движущиеся объекты.
- Производить вычисления. Например, в онлайн-калькуляторе на сайте. Это можно делать и на стороне сервера, и в самом браузере.
- Писать серверную часть сайта или приложения.

Современный JavaScript – это «безопасный» язык программирования. Он не предоставляет низкоуровневый доступ к памяти или процессору, потому что изначально был создан для браузеров, не требующих этого.

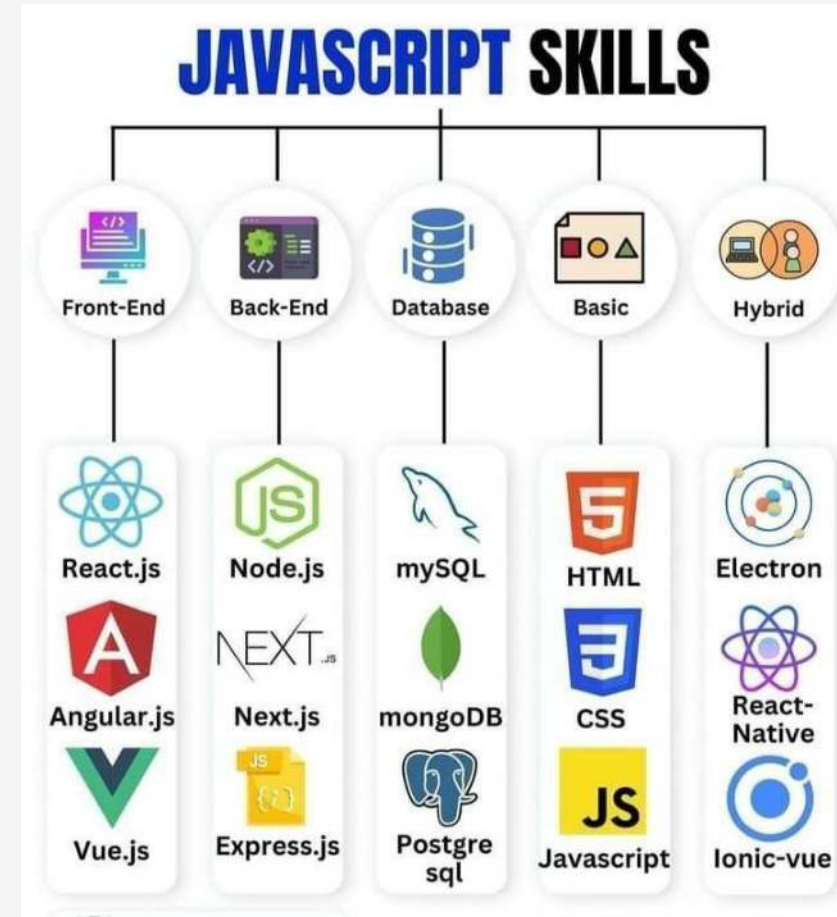
Возможности JavaScript сильно зависят от окружения, в котором он работает. Например, Node.js поддерживает функции чтения/записи произвольных файлов, выполнения сетевых запросов и т.д.

В браузере для JavaScript доступно всё, что связано с манипулированием веб-страницами, взаимодействием с пользователем и веб-сервером.



## Что может JavaScript в браузере?

- Добавлять новый HTML-код на страницу, изменять существующее содержимое, модифицировать стили.
- Реагировать на действия пользователя, щелчки мыши, перемещения указателя, нажатия клавиш.
- Отправлять сетевые запросы на удалённые сервера, скачивать и загружать файлы (технологии AJAX и COMET).
- Получать и устанавливать куки, задавать вопросы посетителю, показывать сообщения.
- Запоминать данные на стороне клиента («local storage»).



## Что может JavaScript в браузере?

# Что не может JavaScript в браузере?

Возможности JavaScript в браузере ограничены ради безопасности пользователя.

Цель заключается в предотвращении доступа недобросовестной веб-страницы к личной информации или нанесения ущерба данным пользователя.

**1** JavaScript на веб-странице не может читать/записывать произвольные файлы на жёстком диске, копировать их и запускать программы. Он не имеет прямого доступа к системным функциям ОС.

Современные браузеры позволяют ему работать с файлами, но с ограниченным доступом, и предоставляют его, только если пользователь выполняет определённые действия, такие как «перетаскивание» файла в окно браузера или его выбор с помощью тега `<input>`.

Существуют способы взаимодействия с камерой/микрофоном и другими устройствами, но они требуют явного разрешения пользователя.

# Что не может JavaScript в браузере?

2 Различные окна/вкладки не знают друг о друге. Иногда одно окно, используя JavaScript, открывает другое окно. Но даже в этом случае JavaScript с одной страницы не имеет доступа к другой, если они пришли с разных сайтов (с другого домена, протокола или порта).

Это называется «Политика одинакового источника» (Same Origin Policy). Чтобы обойти это ограничение, обе страницы должны согласиться с этим и содержать JavaScript-код, который специальным образом обменивается данными.

Это ограничение необходимо, опять же, для безопасности пользователя. Страница <https://anysite.com>, которую открыл пользователь, не должна иметь доступ к другой вкладке браузера с URL <https://gmail.com> и воровать информацию оттуда.

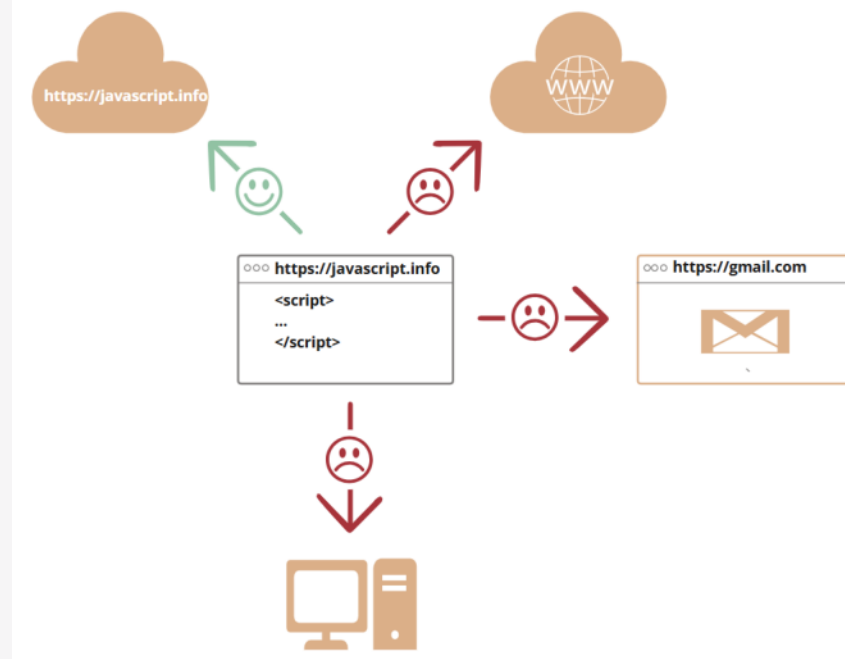


# Что не может JavaScript в браузере?

3 JavaScript может легко взаимодействовать с сервером, с которого пришла текущая страница. Но его способность получать данные с других сайтов/доменов ограничена. Хотя это возможно в принципе, для чего требуется явное согласие (выраженное в заголовках HTTP) с удалённой стороной. Опять же, это ограничение безопасности.

Подобные ограничения не действуют, если JavaScript используется вне браузера, например — на сервере.

Современные браузеры предоставляют плагины/расширения, с помощью которых можно запрашивать дополнительные разрешения.



Как минимум, *три* сильные стороны JavaScript:

- Полная интеграция с HTML/CSS.
- Простые вещи делаются просто.
- Поддерживается всеми основными браузерами и включён по умолчанию.

JavaScript – это единственная браузерная технология, сочетающая в себе все эти три вещи.

Вот что делает JavaScript особенным. Вот почему это самый распространённый инструмент для создания интерфейсов в браузере.

Хотя, конечно, JavaScript позволяет делать приложения не только в браузерах, но и на сервере, на мобильных устройствах и т.п.

## Что делает JavaScript особенным?



2

# ПЕРЕМЕННЫЕ

# Переменные в JavaScript

JavaScript-приложению нужно работать с информацией.

Например:

– Интернет-магазин – информация может включать продаваемые товары и корзину покупок.

– Чат – информация может включать пользователей, сообщения и многое другое.

Переменные используются для хранения этой информации.

Переменная – это «именованное хранилище» для данных. Мы можем использовать переменные для хранения товаров, посетителей и других данных.

Для создания переменной в JavaScript используйте ключевое слово `let`. Приведённая ниже инструкция создаёт (другими словами, объявляет) переменную с именем "message".

```
let message;
```

Теперь можно поместить в неё данные (другими словами, определить переменную), используя оператор присваивания `=`

```
let message;
```

```
message = 'Hello'; // сохранить строку 'Hello' в переменной с именем message
```

## Объявление нескольких переменных

Мы также можем объявить несколько переменных в одной строке:

```
let user = 'Artur', age = 25, message = 'Hi';
```

Такой способ может показаться короче, но мы не рекомендуем его. Для лучшей читаемости объявляйте каждую переменную на новой строке.

Многострочный вариант немного длиннее, но легче для чтения:

Некоторые люди также определяют несколько переменных в многострочном стиле:

Все эти варианты работают одинаково. Так что это вопрос личного вкуса и эстетики.

```
let user = 'Artur';  
let age = 25;  
let message = 'Hi';
```

```
let user = 'Artur'  
  age = 25,  
  message = 'Hi';
```

## Концепция переменной

Мы легко поймём концепцию «переменной», если представим её в виде «коробки» для данных с уникальным названием на ней.

Например, переменную `message` можно представить как коробку с названием `"message"` и значением `"Hello!"` внутри.



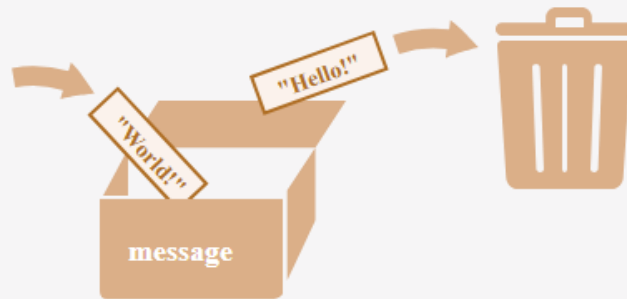
Мы можем положить любое значение в коробку.

Также можем изменить его столько раз, сколько захотим.

```
let message;  
  
message = 'Hello!';  
  
message = 'World!'; // значение изменено  
  
alert(message);
```

## Концепция переменной

При изменении значения старые данные удаляются из переменной.



Мы также можем объявить две переменные и скопировать данные из одной в другую:

```
let hello = 'Hello world!';  
  
let message;  
  
// копируем значение 'Hello world' из переменной hello в переменную message  
message = hello;  
  
// теперь две переменные содержат одинаковые данные
```



Переменная может быть объявлена только один раз.

Повторное объявление той же переменной является ошибкой

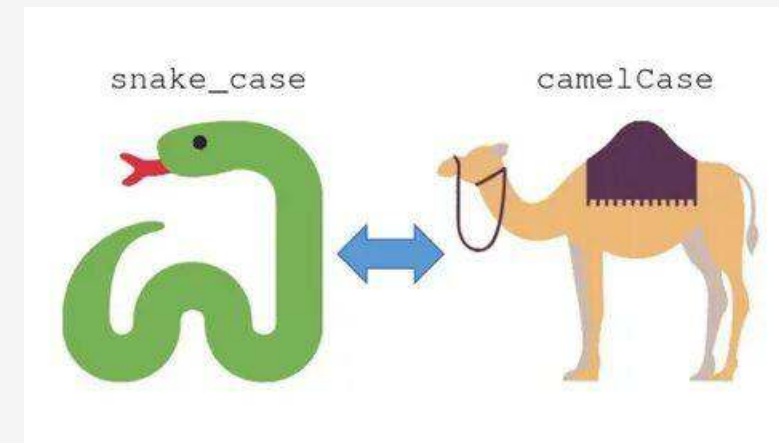
## Имена переменных

В JavaScript есть два ограничения, касающиеся имён переменных:

1. Имя переменной должно содержать только буквы, цифры или символы \$ и \_.
2. Первый символ не должен быть цифрой.

Если имя содержит несколько слов, обычно используется верблюжья нотация, то есть, слова следуют одно за другим, где каждое следующее слово начинается с заглавной буквы: `myVeryLongName`.

Знак доллара '\$' и подчёркивание '\_' также можно использовать в названиях. Это обычные символы, как и буквы, без какого-либо особого значения.





## Имена переменных

И еще несколько важных моментов:

- Переменные с именами `apple` и `APPLE` – это две разные переменные.
- Можно использовать любой язык, включая кириллицу или даже иероглифы.
- Существует список зарезервированных слов\*, которые нельзя использовать в качестве имён переменных, потому что они используются самим языком.

Например: `let`, `class`, `return` и `function` зарезервированы

\*Ознакомиться можно тут: **JavaScript Reserved Words** [https://www.w3schools.com/js/js\\_reserved.asp](https://www.w3schools.com/js/js_reserved.asp)

## Константы

Чтобы объявить константную, то есть, неизменяемую переменную, используйте `const` вместо `let`

Переменные, объявленные с помощью `const`, называются «константами». Их нельзя изменить.

Попытка сделать это приведёт к ошибке

```
const myBirthday = '11.11.1999';  
myBirthday = '07.07.2007'; // ошибка, константу нельзя перезаписать!
```

Широко распространена практика использования констант в качестве псевдонимов для трудно запоминаемых значений, известные до начала исполнения скрипта.

Названия таких констант пишутся с использованием заглавных букв и подчёркивания.

Например, сделаем константы для различных цветов в «шестнадцатеричном формате»

```
const COLOR_RED = "■#F00";  
const COLOR_GREEN = "■#0F0";  
const COLOR_BLUE = "■#00F";  
const COLOR_ORANGE = "■#FF7F00";  
  
// ...когда нам нужно выбрать цвет  
let color = COLOR_ORANGE;  
alert(color); // ■#FF7F00
```

3

# ТИПЫ ДАННЫХ

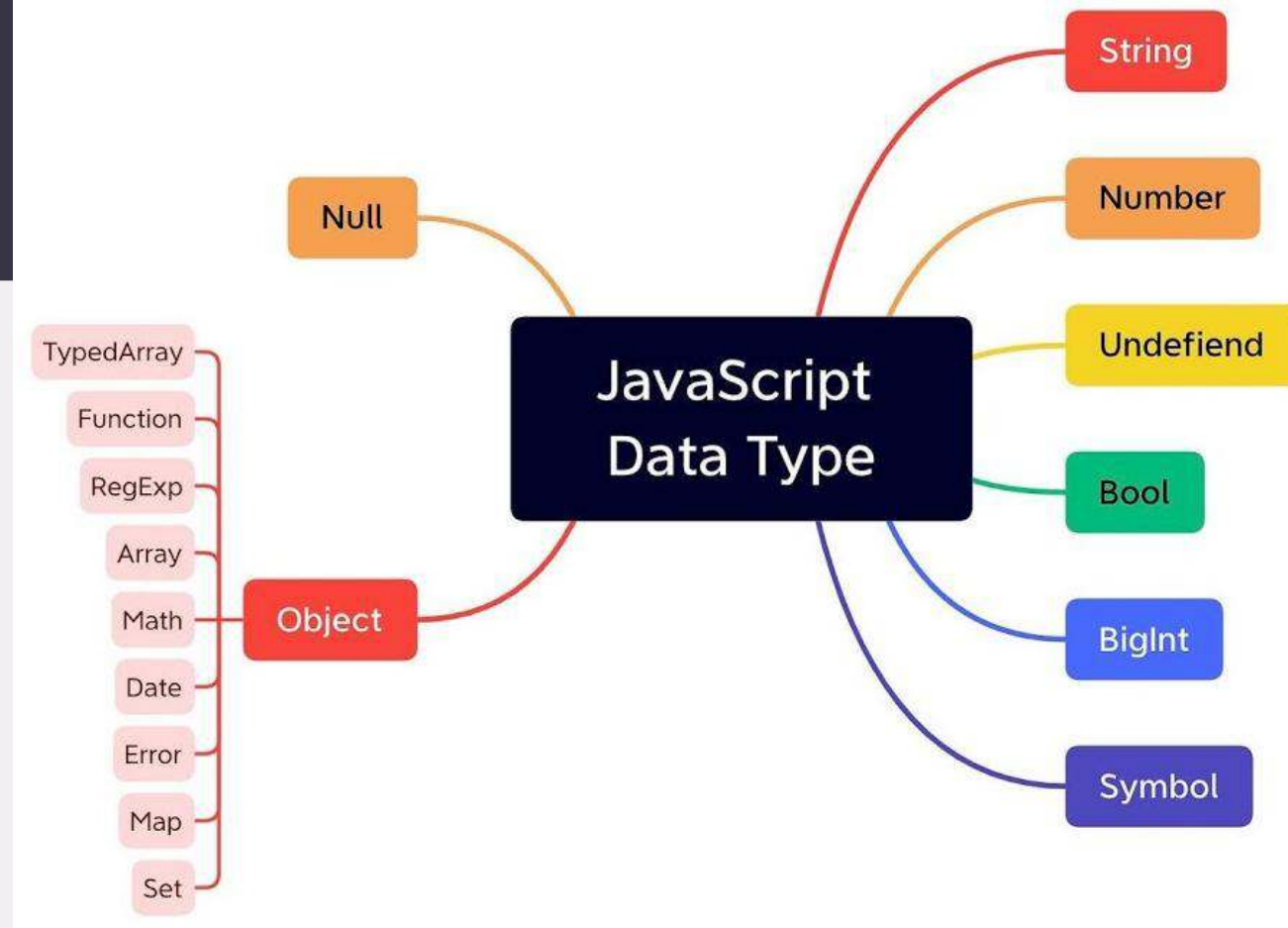
# Типы данных

Значение в JavaScript всегда относится к данным определённого типа. Например, это может быть строка или число.

Есть восемь основных типов данных в JavaScript. В этом вопросе мы рассмотрим их в общем, а потом поговорим подробнее о каждом.

Переменная в JavaScript может содержать любые данные. В один момент там может быть строка, а в другой – число.

Языки программирования, в которых такое возможно, называются «динамически типизированными». Это значит, что типы данных есть, но переменные не привязаны ни к одному из них.



```
// Не будет ошибкой
let message = "hello";
message = 123456;
```

# Типы данных. Типизация

Языки программирования, в которых такое возможно, называются **«динамически типизированными»**.

Это значит, что типы данных есть, но переменные не привязаны ни к одному из них.

```
> '5' - 3
< 2           // слабая типизация + приведение типов = головная боль

> '5' + 3
< '53'        // потому что все мы любим постоянство

> '5' - '4'
< 1           // строка - строка = число. Что?

> '5' + + '5'
< '55'        // окей, допустим

> 'foo' + + 'foo'
< 'fooNaN'    // чудесно!

> '5' + - '2'
< '5-2'       // хорошо

> '5' + - + - - + - - + + - + - + - - - '-2'
< '52'        // будем считать, что так и должно быть
```

## Число

```
let n = 123;  
n = 12.345;
```

Числовой тип данных (number) представляет как целочисленные значения, так и числа с плавающей точкой. Существует множество операций для чисел, например, умножение \*, деление /, сложение +, вычитание - и так далее.

```
alert( 1 / 0 ); // Infinity  
alert( "не число" / 2 ); // NaN, такое деление является ошибкой
```

```
alert( NaN + 1 ); // NaN  
alert( 3 * NaN ); // NaN  
alert( "не число" / 2 - 1 ); // NaN
```

Кроме обычных чисел, существуют так называемые «специальные числовые значения», которые относятся к этому типу данных: Infinity, -Infinity и NaN.

Infinity представляет собой математическую бесконечность  $\infty$ . Это особое значение, которое больше любого числа. Можем получить его в результате деления на ноль или задаем его явно

NaN – вычислительная ошибку. Это результат неправильной или неопределённой математической операции. NaN «прилипчиво». Любая математическая операция с NaN возвращает NaN.

# Математические операции в JS «безопасны»

Мы можем делать что угодно: делить на ноль, обращаться с нечисловыми строками как с числами и т.д. Скрипт никогда не остановится с фатальной ошибкой (не «умрёт»). В худшем случае мы получим NaN как результат выполнения.

Специальные числовые значения относятся к типу «число». Конечно, это не числа в привычном значении этого слова.

```
5 == 5    //true
5 == '5'  //true
5 === 5   //true
5 === '5' //false
```

```
5 != 5    //false
5 != '5'  //false
5 !== 5   //false
5 !== '5' //true
```

```
2 + 2    // 4
4 - 2    // 2
2 * 3    // 6
4 / 2    // 2
```

```
let a = 10; log(++a, a); // 11 11
let b = 10; log(b++, b); // 10 11
let c = 10; log(--c, c); // 9 9
let d = 10; log(d--, d); // 10 9
```

## JAVASCRIPT OPERATOR CHEATSHEET

```
let a = 20; a += 5; // a = 25
let b = 20; b -= 5; // b = 15
let c = 20; c *= 5; // c = 100
let d = 20; d /= 5; // d = 4
let e = 20; e %= 5; // e = 0
```

```
(5 === 5) ? 'a' : 'b' // a
3 > 2 && 1 < 2        // true
3 > 2 || 5 < 2         // true
!true                 // false
```

```
3 > 3    // false
3 < 3    // false
3 >= 3   // true
3 <= 3   // true
5 % 2    // 1
```

# BigInt

В JavaScript тип `number` не может безопасно работать с числами, большими, чем  $(2^{53} - 1)$  (т. е. 9007199254740991) или меньшими, чем  $-(2^{53} - 1)$  для отрицательных чисел.

Если говорить совсем точно, то, технически, тип `number` может хранить большие целые числа (до  $1,7976931348623157 * 10^{308}$ ), но за пределами безопасного диапазона целых чисел  $\pm(2^{53} - 1)$  будет ошибка точности, так как не все цифры помещаются в фиксированную 64-битную память. Поэтому можно хранить «приблизительное» значение.

Например, представленные в листинге два числа ниже совпадают.

```
console.log(9007199254740991 + 1); // 9007199254740992
console.log(9007199254740991 + 2); // 9007199254740992
```



# BigInt

То есть все нечетные целые числа, большие чем  $(2^{53}-1)$ , вообще не могут храниться в типе `number`.

Тип `BigInt` был добавлен в JavaScript, чтобы дать возможность работать с целыми числами произвольной длины.

Чтобы создать значение типа `BigInt`, необходимо добавить `n` в конец числового литерала.

```
// символ "n" в конце означает, что это BigInt  
const bigInt = 1234567890123456789012345678901234567890n;
```

Необходимость в использовании `BigInt`-чисел появляется достаточно редко.

В данный момент `BigInt` поддерживается только в браузерах Firefox, Chrome, Edge и Safari, но не поддерживается в IE.

# Булевый (логический) тип

Булевый тип (boolean) может принимать только два значения: true (истина) и false (ложь).

```
let nameFieldChecked = true; // да, поле отмечено  
let ageFieldChecked = false; // нет, поле не отмечено
```

Такой тип, как правило, используется для хранения значений да/нет: true значит «да, правильно», а false значит «нет, неправильно».

Булевы значения также могут быть результатом сравнений.

```
let isGreater = 4 > 1;  
  
alert( isGreater ); // true (результатом сравнения будет "да")
```

# Значение "null"

```
let age = null;
```

Специальное значение `null` не относится ни к одному из типов, описанных выше. Оно формирует отдельный тип, который содержит только значение `null`.

В JavaScript `null` не является «ссылкой на несуществующий объект» или «нулевым указателем», как в некоторых других языках.

Это просто специальное значение, которое представляет собой «ничего», «пусто» или «значение неизвестно».

**undefined**

**null**

has not been assigned

could be assigned

`typeof undefined`

`typeof object`

`undefined == null //true`

`undefined === null //false`

# Значение "undefined"

```
let age;
```

```
alert(age); // выведет "undefined"
```

Специальное значение `null` не относится ни к одному из типов, описанных выше. Оно формирует отдельный тип, который содержит только значение `null`.

В JavaScript `null` не является «ссылкой на несуществующий объект» или «нулевым указателем», как в некоторых других языках.

Это просто специальное значение, которое представляет собой «ничего», «пусто» или «значение неизвестно».

Обычно `null` используется для присвоения переменной «пустого» или «неизвестного» значения, а `undefined` - для проверок, была ли переменная назначена.

# Объекты и символы

Тип `object` (объект) - особенный .

Все остальные типы называются «примитивными», потому что их значениями могут быть только простые значения (будь то строка, или число, или что-то ещё). В объектах же хранят коллекции данных или более сложные структуры.

Тип `symbol` (символ) используется для создания уникальных идентификаторов в объектах. Мы упоминаем здесь о нём для полноты картины, изучим этот тип после объектов.

# Оператор typeof

Оператор `typeof` возвращает тип аргумента. Это полезно, когда мы хотим обрабатывать значения различных типов по-разному или просто хотим сделать проверку.

У него есть две синтаксические формы:

Если передается выражение, то нужно заключать его в скобки, т.к. `typeof` имеет более высокий приоритет, чем бинарные операторы.

Другими словами, скобки необходимы для определения типа значения, которое получилось в результате выполнения выражения в них.

```
// Обычный синтаксис
typeof 5 // Выведет "number"

// Синтаксис, напоминающий вызов функции (встречается реже)
typeof(5) // Также выведет "number"
```

# Оператор typeof

Вызов `typeof x` возвращает строку с именем в формате:

Последние три строки нуждаются в пояснении:

1. `Math` — это встроенный объект, который предоставляет мат-е операции и константы. Здесь он служит лишь примером объекта.
2. Результатом вызова `typeof null` является `"object"`. Это официально признанная ошибка в `typeof`, ведущая начало с времён создания JavaScript и сохранённая для совместимости. Конечно, `null` не является объектом. Это специальное значение с отдельным типом.
3. Вызов `typeof alert` возвращает `"function"`, тк `alert` является функцией. Изучим функции далее и увидим, что в JS нет специального типа `"функция"`. Функции относятся к объектному типу. Но `typeof` обрабатывает их особым образом, возвращая `"function"`. Так тоже повелось от создания JavaScript. Формально это неверно, но может быть удобным на практике.

```
typeof undefined // "undefined"

typeof 0 // "number"

typeof 10n // "bigint"

typeof true // "boolean"

typeof "foo" // "string"

typeof Symbol("id") // "symbol"

typeof Math // "object" (1)

typeof null // "object" (2)

typeof alert // "function" (3)
```

# Сводная информация по типам данных

| Тип данных       | Пример                          | Описание                                                                                     |
|------------------|---------------------------------|----------------------------------------------------------------------------------------------|
| <b>Number</b>    | 42, 3.14, NaN, Infinity         | Числовой тип. Поддерживает целые и вещественные числа, а также спец. значения NaN, Infinity. |
| <b>BigInt</b>    | 123n,<br>9007199254740991n      | Для работы с очень большими целыми числами, выходящими за пределы Number.                    |
| <b>String</b>    | 'hello', "world",<br>`template` | Строки текста. Поддерживаются шаблонные строки с подстановкой переменных.                    |
| <b>Boolean</b>   | true, false                     | Логический тип: истина или ложь.                                                             |
| <b>Null</b>      | null                            | Специальное значение: «ничего» или «пустое значение».                                        |
| <b>Undefined</b> | undefined                       | Переменная объявлена, но не имеет значения.                                                  |
| <b>Symbol</b>    | Symbol('id')                    | Уникальный и неизменяемый идентификатор. Используется для ключей объектов.                   |
| <b>Object</b>    | {}, [], function() {}           | Сложный тип данных: коллекции, массивы, функции, даты и т.д.                                 |



4

# ДИАЛОГОВЫЕ ОКНА В JAVASCRIPT

# Диалоговые окна в JavaScript

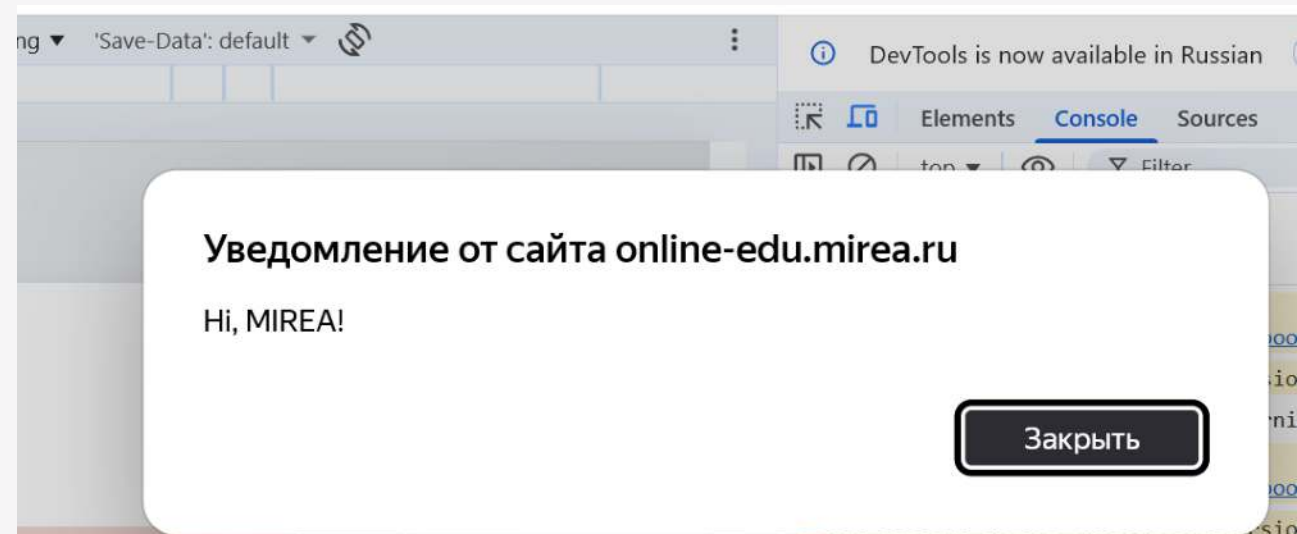
## alert

```
> alert("Hi, MIREA!");  
>
```

С этой функцией мы уже знакомы. Она показывает сообщение и ждёт, пока пользователь нажмёт кнопку «ОК».

Это небольшое окно с сообщением называется модальным окном. Понятие модальное означает, что пользователь не может взаимодействовать с интерфейсом остальной части страницы, нажимать на другие кнопки и т.д. до тех пор, пока взаимодействует с окном.

В данном случае – пока не будет нажата кнопка «ОК».



Функция `prompt` принимает два аргумента.

`Prompt` отобразит модальное окно с текстом, полем для ввода текста и кнопками ОК/Отмена.

- `title`: Текст для отображения в окне.
- `default`: Необязательный второй параметр, который устанавливает начальное значение в поле для текста в окне.

Квадратные скобки вокруг `default` в описанном выше синтаксисе означают, что параметр факультативный, необязательный.

```
result = prompt(title, [default]);
```

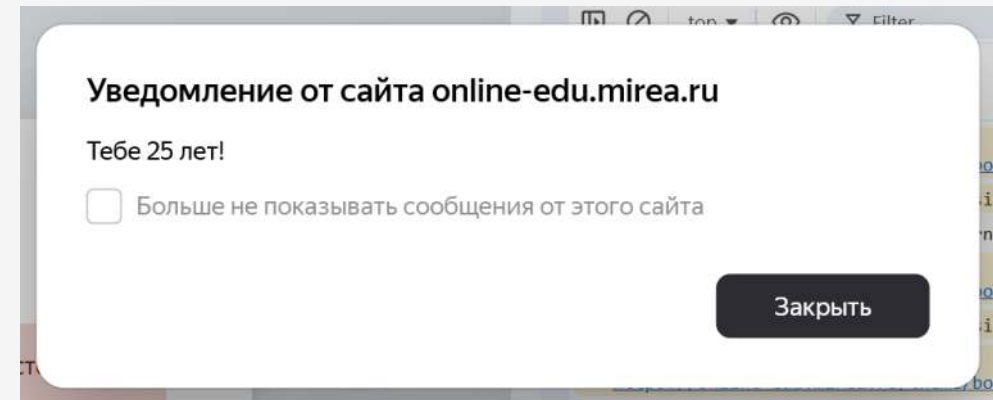
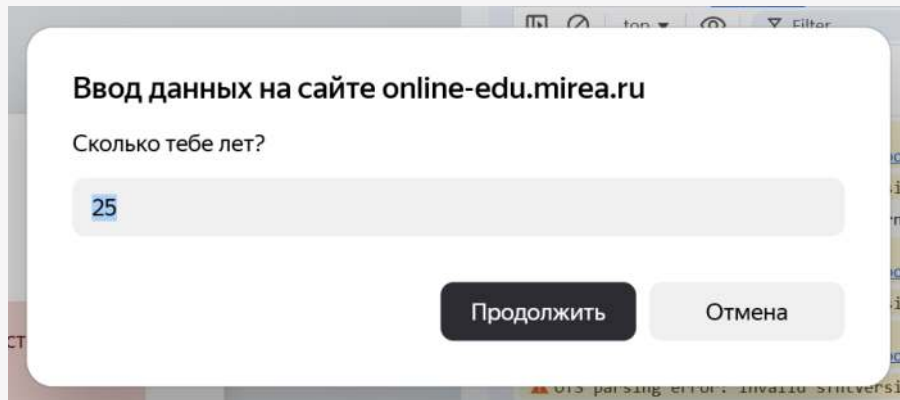
# prompt

```
> let age = prompt('Сколько тебе лет?', 25);  
  
alert(`Тебе ${age} лет!`); // Тебе 25 лет!
```

## Диалоговые окна в JavaScript

Пользователь может напечатать что-либо в поле ввода и нажать ОК. Введённый текст будет присвоен переменной `result`. Пользователь также может отменить ввод нажатием на кнопку «Отмена» или нажав на клавишу `key:Esc`. В этом случае значением `result` станет `null`.

Вызов `prompt` возвращает текст, указанный в поле для ввода, или `null`, если ввод отменён пользователем.



# prompt

```
> let age = prompt('Сколько тебе лет?', 25);  
  
alert(`Тебе ${age} лет!`); // Тебе 25 лет!
```

## Диалоговые окна в JavaScript

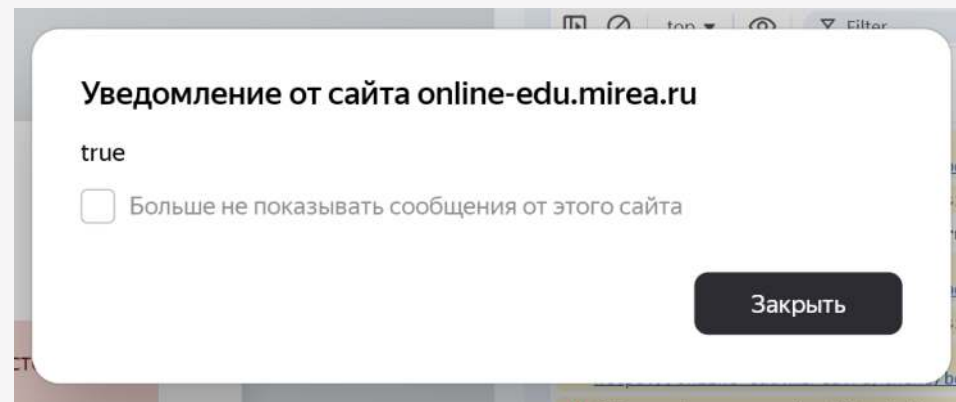
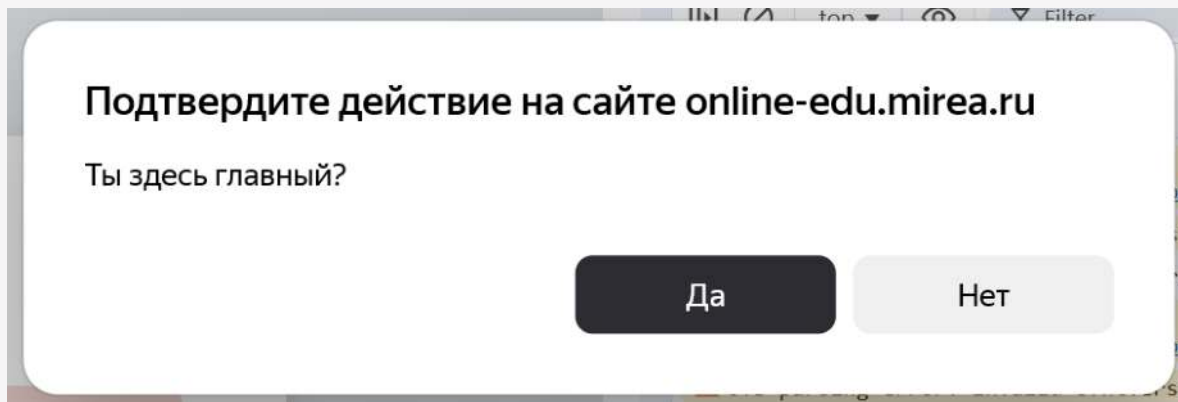
# Диалоговые окна в JavaScript

## confirm

```
> let isBoss = confirm("Ты здесь главный?");  
  
alert( isBoss ); // true, если нажата ОК
```

Функция `confirm` отображает модальное окно с текстом вопроса `question` и двумя кнопками: ОК и Отмена.

Результат – `true`, если нажата кнопка ОК. В других случаях – `false`.



5

# ПРЕОБРАЗОВАНИЕ ТИПОВ

# Строковое преобразование

Строковое преобразование происходит, когда требуется представление чего-либо в виде строки. Например, `alert(value)` преобразует значение к строке.

Также мы можем использовать функцию `String(value)`, чтобы преобразовать значение к строке.

Преобразование происходит очевидным образом. `false` становится `"false"`, `null` становится `"null"`.

```
let value = true;
alert(typeof value); // boolean

value = String(value); // теперь value это строка "true"
alert(typeof value); // string
```

# Численное преобразование

```
alert( "6" / "2" ); // 3, строки преобразуются в числа
```

Численное преобразование происходит в математических функциях и выражениях.

Мы можем использовать функцию `Number(value)`, чтобы явно преобразовать `value` к числу.

Явное преобразование часто применяется, когда мы ожидаем получить число из строкового контекста, например из текстовых полей форм.

Если строка не может быть явно приведена к числу, то результатом преобразования будет `NaN`.

```
let str = "123";  
alert(typeof str); // string  
  
let num = Number(str); // становится числом 123  
  
alert(typeof num); // number
```

```
let age = Number("Любая строка вместо числа");  
  
alert(age); // NaN, преобразование не удалось
```



# Правила численного преобразования

`null` и `undefined` ведут себя по-разному. Так, `null` становится нулём, тогда как `undefined` приводится к `NaN`.

Большинство математических операторов также производит данное преобразование, как мы увидим далее.

| Значение                  | Преобразуется в                                                                                                                                                                                                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>undefined</code>    | <code>NaN</code>                                                                                                                                                                                                                                                                       |
| <code>null</code>         | <code>0</code>                                                                                                                                                                                                                                                                         |
| <code>true / false</code> | <code>1 / 0</code>                                                                                                                                                                                                                                                                     |
| <code>string</code>       | Пробельные символы (пробелы, знаки табуляции <code>¥t</code> , знаки новой строки <code>¥n</code> и т. п.) по краям обрезаются. Далее, если остаётся пустая строка, то получаем <code>0</code> , иначе из непустой строки «считывается» число. При ошибке результат <code>NaN</code> . |

# Логическое преобразование

Логическое преобразование самое простое.

Происходит в логических операциях (позже мы познакомимся с условными проверками и подобными конструкциями), но также может быть выполнено явно с помощью функции `Boolean(value)`.

Правило преобразования:

- Значения, которые интуитивно «пустые», вроде 0, пустой строки, `null`, `undefined` и `NaN`, становятся `false`.
- Все остальные значения становятся `true`.

```
alert( Boolean(1) ); // true
alert( Boolean(0) ); // false
```

```
alert( Boolean("Привет!") ); // true
alert( Boolean("") ); // false
```

```
alert( Boolean("0") ); // true
alert( Boolean(" ") ); // пробел это тоже true (любая непустая строка это true)
```

## Итого

Существует 3 наиболее широко используемых преобразования:

- Строковое – происходит, когда нам нужно что-то вывести. Может быть вызвано с помощью `String(value)`. Для примитивных значений работает очевидным образом.

- Численное – происходит в математических операциях. Может быть вызвано с помощью `Number(value)`.

- Логическое – происходит в логических операциях. Может быть вызвано с помощью `Boolean(value)`.

Большую часть из этих правил легко понять и запомнить.

Особые случаи, в которых часто допускаются ошибки:

- `undefined` при численном преобразовании становится `NaN`, не `0`.

- `"0"` и строки из одних пробелов типа `" "` при логическом преобразовании всегда `true`.

6

# БАЗОВЫЕ МАТЕМАТИЧЕСКИЕ ОПЕРАЦИИ

# Термины: «унарный», «бинарный», «операнд»

– Операнд - то, к чему применяется оператор. Например, в умножении  $5 * 2$  есть два операнда: левый операнд равен 5, а правый операнд равен 2.

Иногда их называют «аргументами» вместо «операндов».

– Унарным называется оператор, который применяется к одному операнду. Например, оператор унарный минус "-" меняет знак числа на противоположный.

```
let x = 1;
```

```
x = -x;
```

```
alert( x ); // -1, применили унарный минус
```

– Бинарным называется оператор, который применяется к двум операндам. Тот же минус существует и в бинарной форме.

```
let x = 1, y = 3;
```

```
alert( y - x ); // 2, бинарный минус вычитает значения
```

# Базовые математические операции

## Математика

| Оператор | Название                     | Пример                               |
|----------|------------------------------|--------------------------------------|
| +        | Сложение                     | $x + y$                              |
| -        | Вычитание                    | $x - y$                              |
| *        | Умножение                    | $x * y$                              |
| /        | Деление                      | $x / y$                              |
| %        | Остаток от деления           | $x \% y$                             |
| ++       | Инкремент (увеличивает на 1) | <code>++x</code> or <code>x++</code> |
| --       | Декремент ( уменьшает на 1)  | <code>--x</code> or <code>x--</code> |
| **       | Возведение в степень         | $x ** y$                             |

# Приоритет операторов

В том случае, если в выражении есть несколько операторов – порядок их выполнения определяется приоритетом, или, другими словами, существует определённый порядок выполнения операторов.

Из школы мы знаем, что умножение в выражении  $1 + 2 * 2$  выполнится раньше сложения. Это как раз и есть «приоритет». Говорят, что умножение имеет более высокий приоритет, чем сложение.

Скобки важнее, чем приоритет, так что, если мы не удовлетворены порядком по умолчанию, мы можем использовать их, чтобы изменить приоритет. Например, написать  $(1 + 2) * 2$ .

В JavaScript много операторов. Каждый оператор имеет соответствующий номер приоритета. Тот, у кого это число больше, – выполнится раньше. Если приоритет одинаковый, то порядок выполнения – слева направо.

## Базовые математические операции

# Приоритеты операторов в JavaScript

| Приоритет | Тип оператора                     | Ассоциативность      | Конкретные операторы                                                                             |
|-----------|-----------------------------------|----------------------|--------------------------------------------------------------------------------------------------|
| 18        | Группировка                       | –                    | ( ... )                                                                                          |
| 17        | Доступ к свойствам                | слева направо        | ... . ... , ... [ ... ] , ... ( ... ) , new ... ( ... ) , ?.                                     |
| 16        | new (без списка аргументов)       | справа налево        | new ...                                                                                          |
| 15        | Постфиксный инкремент / декремент | –                    | ... ++ , ... --                                                                                  |
| 14        | Унарные операторы                 | справа налево        | ! ... , ~ ... , + ... , - ... , ++ ... , -- ... , typeof ... , void ... , delete ... , await ... |
| 13        | Возведение в степень              | справа налево        | ... ** ...                                                                                       |
| 12        | Умножение / Деление / Остаток     | слева направо        | ... * ... , ... / ... , ... % ...                                                                |
| 11        | Сложение / Вычитание              | слева направо        | ... + ... , ... - ...                                                                            |
| 10        | Побитовые сдвиги                  | слева направо        | ... << ... , ... >> ... , ... >>> ...                                                            |
| 9         | Сравнения                         | слева направо        | < , <= , > , >= , in , instanceof                                                                |
| 8         | Равенство                         | слева направо        | == , != , === , !==                                                                              |
| 7         | <b>Побитовое «И»</b>              | <b>слева направо</b> | <b>... &amp; ...</b>                                                                             |
| 6         | Побитовое исключаящее «ИЛИ»       | слева направо        | ... ^ ...                                                                                        |
| 5         | Побитовое «ИЛИ»                   | слева направо        | ...   ...                                                                                        |
| 4         | Логическое «И»                    | слева направо        | ... && ...                                                                                       |
| 3         | Логическое «ИЛИ», нулевое слияние | слева направо        | ...    ...                                                                                       |
| 2         | Присваивание / Тернарный / yield  | справа налево        | = , += , -= , **= , *= , /= , %= , <=< , >>= , >>>= , &= , ^= , `                                |
| 1         | Запятая (последовательность)      | слева направо        | ... , ...                                                                                        |



# Приоритеты операторов в JavaScript

- Чем выше число приоритета, тем раньше выполняется оператор.
- Операторы с одинаковым приоритетом выполняются в порядке, указанном в колонке «Ассоциативность».
- Ассоциативность может быть слева направо (обычно) или справа налево (например, присваивание, унарные операторы).

Так как «унарный плюс» имеет приоритет 14, который выше, чем 11 у «сложения» (бинарный плюс), то в выражении "+apples + +oranges" сначала выполнятся унарные плюсы, а затем сложение.

*сейчас*



*потом*



# Присваивание и присваивание по цепочке

Большинство операторов в JavaScript возвращают значение. Для некоторых это очевидно, например сложение `+` или умножение `*`. Но и присваивания не является исключением.

Вызов `x = value` записывает `value` в `x` и возвращает его.

Благодаря этому присваивание можно использовать как часть более сложного выражения.

В примере выше результатом `(a = b + 1)` будет значение, которое присваивается переменной `a` (то есть 3). Потом оно используется для дальнейших вычислений.

Такое присваивание работает справа налево. Сначала вычисляется самое правое выражение `2 + 2`, и затем результат присваивается переменным слева:

`c`, `b` и `a`. В конце у всех переменных будет одно значение

```
let x = 2 * 2 + 1;  
  
alert( x ); // 5
```

```
let a = 1;  
let b = 2;  
  
let c = 3 - (a = b + 1);  
  
alert( a ); // 3  
alert( c ); // 0
```

# Сокращенная арифметика

Часто нужно применить оператор к переменной и сохранить результат в ней же.

```
let n = 2;  
n = n + 5;  
n = n * 2;
```

Эту запись можно укоротить при помощи совмещённых операторов += и \*=

```
let n = 2;  
n += 5; // теперь n = 7 (работает как n = n + 5)  
n *= 2; // теперь n = 14 (работает как n = n * 2)  
  
alert( n ); // 14
```

Подобные краткие формы записи существуют для всех арифметических и побитовых операторов: /=, -=, \*\*= и так далее.

Вызов с присваиванием имеет в точности такой же приоритет, как обычное присваивание, то есть выполнится после большинства других операций

# Инкремент/декремент

Одной из наиболее частых числовых операций является увеличение или уменьшение на единицу.

Для этого существуют даже специальные операторы:

- Инкремент ++ увеличивает переменную на 1.
- Декремент -- уменьшает переменную.

Инкремент/декремент можно применить только к переменной. Попытка использовать его на значении, типа 5++, приведёт к ошибке.

Операторы ++ и -- могут быть расположены не только после, но и до переменной.

- Когда оператор идёт после переменной – это «постфиксная форма»: counter++.
- «Префиксная форма» – когда оператор идёт перед переменной: ++counter.

Обе эти инструкции делают одно и то же: увеличивают counter на 1.

| ОПЕРАТОР     | ОПИСАНИЕ                          | ПРИМЕР                    |
|--------------|-----------------------------------|---------------------------|
| ++ инкремент | Увеличивает переменную на единицу | a = 1<br>a++/++a<br>a = 2 |
| -- декремент | Уменьшает переменную на единицу   | a = 2<br>a--/--a<br>a = 1 |

# Инкремент/декремент

Чтобы увидеть разницу префиксной и постфиксной форм, рассмотрим небольшой пример

```
let counter = 1;  
let a = ++counter; // (*)  
  
alert(a); // *!*2*/!*
```

```
let counter = 1;  
let a = counter++; // (*) меняем ++counter на counter++  
  
alert(a); // *!*1*/!*
```

В строке (\*) префиксная форма ++counter увеличивает counter и возвращает новое значение 2. Так что alert покажет 2.

В строке (\*) 2 скриншота постфиксная форма counter++ также увеличивает counter, но возвращает старое значение (которое было до увеличения). Так что alert покажет 1.

## Побитовые операторы

Побитовые операторы работают с 32-разрядными целыми числами (при необходимости приводят к ним), на уровне их внутреннего двоичного представления.

Эти операторы не являются чем-то специфичным для JavaScript, они поддерживаются в большинстве языков программирования.

Они используются редко, когда возникает необходимость оперировать с числами на очень низком (побитовом) уровне.

| ОПЕРАТОР | ОПИСАНИЕ                          |
|----------|-----------------------------------|
| ~        | побитовое НЕ                      |
| &        | побитовое И                       |
|          | побитовое ИЛИ                     |
| ^        | исключающее побитовое ИЛИ         |
| >>       | сдвиг вправо                      |
| >>>      | сдвиг вправо с заполнением нулями |
| <<       | сдвиг влево                       |

# Оператор запятая

Оператор «запятая» (,) редко применяется и является одним из самых необычных.

Иногда он используется для написания более короткого кода, поэтому нам нужно знать его, чтобы понимать, что при этом происходит.

Оператор «запятая» предоставляет нам возможность вычислять несколько выражений, разделяя их запятой ",". Каждое выражение выполняется, но возвращается результат только последнего.

Первое выражение  $1 + 2$  выполняется, а результат отбрасывается. Затем идёт  $3 + 4$ , выражение выполняется и возвращается результат.

Пожалуйста, обратите внимание, что оператор "," имеет очень низкий приоритет, ниже  $=$ , поэтому скобки важны в приведённом выше примере.

```
let a = (1 + 2, 3 + 4);  
  
alert( a ); // 7 (результат вычисления 3 + 4)
```

# Оператор запятая

Зачем нам оператор, который отбрасывает всё, кроме последнего выражения?

Иногда его используют в составе более сложных конструкций, чтобы сделать несколько действий в одной строке

```
// три операции в одной строке
for (a = 1, b = 3, c = a * b; a < 10; a++) {
  //...
}
```

Такие трюки используются во многих JavaScript-фреймворках. Вот почему мы упоминаем их. Но обычно они не улучшают читабельность кода, поэтому стоит хорошо подумать, прежде чем их использовать.



# 7 ОПЕРАТОРЫ СРАВНЕНИЯ

## Сравнение в JS

Многие операторы сравнения известны нам из математики.

В JavaScript они записываются так:

- Больше/меньше:  $a > b$ ,  $a < b$ .
- Больше/меньше или равно:  $a \geq b$ ,  $a \leq b$ .
- Равно:  $a == b$ . Обратите внимание, для сравнения используется двойной знак равенства `==`.

Один знак равенства  $a = b$  означал бы присваивание.

- Не равно. В математике обозначается символом  $\neq$ , но в JavaScript записывается как  $a \neq b$ .

Все операторы сравнения возвращают значение логического типа: `true` или `false`.

| Operator           | Description                        | Example                  | Returns            |
|--------------------|------------------------------------|--------------------------|--------------------|
| <code>==</code>    | Equal to                           | <code>5 == 10</code>     | <code>false</code> |
| <code>===</code>   | Identical (equal and of same type) | <code>5 === 5</code>     | <code>true</code>  |
| <code>!=</code>    | Not equal                          | <code>5 != 10</code>     | <code>true</code>  |
| <code>&gt;</code>  | Greater than                       | <code>5 &gt; 10</code>   | <code>false</code> |
| <code>&gt;=</code> | Greater than or equal to           | <code>10 &gt;= 5</code>  | <code>true</code>  |
| <code>&lt;</code>  | Less than                          | <code>5 &lt; 10</code>   | <code>true</code>  |
| <code>&lt;=</code> | Less than or equal to              | <code>10 &lt;= 10</code> | <code>true</code>  |

## Сравнение строк

```
alert( 'Я' > 'А' ); // true
alert( 'Коты' > 'Кода' ); // true
alert( 'Сонный' > 'Сон' ); // true
```

Алгоритм сравнения двух строк довольно прост:

1. Сначала сравниваются первые символы строк.
2. Если первый символ первой строки больше (меньше), чем первый символ второй, то первая строка больше (меньше) второй. Сравнение завершено.
3. Если первые символы равны, то таким же образом сравниваются уже вторые символы строк.
4. Сравнение продолжается, пока не закончится одна из строк.
5. Если обе строки заканчиваются одновременно, то они равны. Иначе, большей считается более длинная строка.

В примере сравнение 'Я' > 'А' завершится на первом шаге, тогда как строки 'Коты' и 'Кода' будут сравниваться посимвольно:

1. К равна К.
2. о равна о.
3. т больше, чем д. На этом сравнение заканчивается. Первая строка больше.

# Сравнение строк и типов

В JavaScript имеет значение регистр символов. Заглавная буква «А» не равна строчной «а». Какая же из них больше? Строчная «а». Почему? Потому что строчные буквы имеют больший код во внутренней таблице кодирования, которую использует JavaScript (Unicode).

При сравнении значений разных типов JavaScript приводит каждое из них к числу.

Логическое значение true становится 1, а false – 0.

```
alert( true == 1 ); // true  
alert( false == 0 ); // true
```

```
alert( '2' > 1 ); // true, строка '2' становится числом 2  
alert( '01' == 1 ); // true, строка '01' становится числом 1
```

# Строгое сравнение

Использование обычного сравнения `==` может вызывать проблемы. Например, оно не отличает `0` от `false`, та же проблема с пустой строкой.

Это происходит из-за того, что операнды разных типов преобразуются оператором `==` к числу. В итоге, и пустая строка, и `false` становятся нулём.

Как же тогда отличать `0` от `false`?

```
alert( '' == false ); // true
alert( 0 == false ); // true
alert( 0 === false ); // false, так как сравниваются разные типы
```

**Оператор строгого равенства `===` проверяет равенство без приведения типов.**

Другими словами, если `a` и `b` имеют разные типы, то проверка `a === b` немедленно возвращает `false` без попытки их преобразования.

## Операторы сравнения

## Сравнения сложнее

### Операторы “==” и “===”

- Алгоритм абстрактного сравнения

```
" == '0'           // false
0 == ""           // true
0 == '0'          // true
false == undefined // false
false == null      // false
null == undefined  // true
```

- Алгоритм строгого сравнения

```
" === '0'          // false
0 === ""           // false
0 === '0'          // false
false === undefined // false
false === null      // false
null === undefined  // false
+0 === -0           // true
```

| x                 | y         | ==   | ===   |
|-------------------|-----------|------|-------|
| undefined         | undefined | true | true  |
| null              | null      | true | true  |
| true              | true      | true | true  |
| false             | false     | true | true  |
| 'foo'             | 'foo'     | true | true  |
| 0                 | 0         | true | true  |
| +0                | -0        | true | true  |
| +0                | 0         | true | true  |
| -0                | 0         | true | true  |
| 0                 | false     | true | false |
| ""                | false     | true | false |
| ""                | 0         | true | false |
| '0'               | 0         | true | false |
| '17'              | 17        | true | false |
| [1, 2]            | '1,2'     | true | false |
| new String('foo') | 'foo'     | true | false |
| null              | undefined | true | false |

## Операторы сравнения

# Как избежать проблем

Зачем мы рассмотрели все эти примеры? Должны ли мы постоянно помнить обо всех этих особенностях? Не обязательно. Со временем все они станут вам знакомы, но можно избежать проблем, если следовать надёжным правилам:

- Относитесь очень осторожно к любому сравнению с `undefined/null`, кроме случаев строгого равенства `===`.
- Не используйте сравнения `>=` `>` `<` `<=` с переменными, которые могут принимать значения `null/undefined`, разве что вы полностью уверены в том, что делаете. Если переменная может принимать эти значения, то добавьте для них отдельные проверки.



```
> null > 0
```

```
< false
```

```
> null < 0
```

```
< false
```

```
> null == 0
```

```
< false
```

```
> null >= 0
```

8

## УСЛОВНОЕ ВЕТВЛЕНИЕ



# Инструкция if

Инструкция `if(...)` вычисляет условие в скобках и, если результат `true`, то выполняет блок кода.

```
let year = prompt('В каком году вышел мультфильм "Тачки"?', '');  
  
if (year == 2006) alert( 'Ты прав!' );
```

В примере выше, условие – это простая проверка на равенство (`year == 2006`), но оно может быть и гораздо более сложным.

Если мы хотим выполнить более одной инструкции, то нужно заключить блок кода в фигурные скобки

```
if (year == 2006) {  
    alert( "Правильно!" );  
    alert( "Кчау!" );  
}
```

Рекомендуется использовать фигурные скобки `{}` всегда, когда используете инструкцию `if`, даже если выполняется только одна команда. Это улучшает читаемость кода.

### Блок "else"

Инструкция if может содержать необязательный блок else («иначе»). Он выполняется, когда условие ложно.

```
let year = prompt('В каком году была опубликована спецификация ECMAScript-2015?', '');

if (year == 2015) {
  alert( 'Да вы знаток!' );
} else {
  alert( 'А вот и неправильно!' ); // любое значение, кроме 2015
}
```

# Несколько условий: else if

Иногда нужно проверить несколько вариантов условия. Для этого используется блок else if

```
let year = prompt('В каком году вышел мультфильм "Тачки"?', '');

if (year < 2006) {
  alert( 'Это слишком рано...' );
} else if (year > 2006) {
  alert( 'Это поздновато' );
} else {
  alert( 'Верно!' );
}
```

В приведённом выше коде JavaScript сначала проверит `year < 2006`. Если это неверно, он переходит к следующему условию `year > 2006`. Если оно тоже ложно, тогда сработает последний alert.

Блоков else if может быть и больше. Присутствие блока else не является обязательным.

# Условный оператор "?"

Иногда нам нужно определить переменную в зависимости от условия.

Так называемый «условный» оператор «вопросительный знак» позволяет нам сделать это более коротким и простым способом.

Оператор представлен знаком вопроса – "?". Его также называют «тернарный», так как этот оператор, единственный в своём роде, имеет три аргумента.

```
let result = условие ? значение1 : значение2;
```

Сначала вычисляется условие: если оно истинно, тогда возвращается значение1, в противном случае -- значение2

# Несколько операторов "?"

Последовательность операторов «вопросительного знака» позволяет вернуть значение, которое зависит от более чем одного условия.

Поначалу может быть сложно понять, что происходит. Но при ближайшем рассмотрении мы видим, что это обычная последовательная проверка:

1. Первый знак вопроса проверяет `age < 16`.
2. Если верно - возвращает 'Приветики!'.

В противном случае, проверяет выражение после двоеточия ':', вычисляет `age < 18`.

3. Если это верно - возвращает 'Привет!'. В противном случае, проверяет выражение после следующего двоеточия ':', вычисляет `age < 100`.
4. Если это верно - возвращает 'Здравствуйте!'. В противном случае, возвращает выражение после последнего двоеточия - 'Какой необычный возраст!'.

```
let age = prompt('Возраст?', 18);

let message = (age < 16) ? 'Приветики!' :
  (age < 18) ? 'Привет!' :
  (age < 100) ? 'Здравствуйте!' :
  'Какой необычный возраст!';

alert( message );
```

9

# ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

## Логические операторы

# Логические операторы в JS

| Оператор            | Обозначение | Пример использования | Операция                                                   |
|---------------------|-------------|----------------------|------------------------------------------------------------|
| Логическое НЕ (NOT) | !           | !x                   | true, если x равен false; или false, если x равен true     |
| Логическое И (AND)  | &&          | x && y               | true, если и x, и y равны true; в противном случае – false |
| Логическое ИЛИ (OR) |             | x    y               | true, если x или y равен true; в противном случае – false  |

В JavaScript есть семь логических операторов:

- || (ИЛИ)
  - о ||= (Оператор логического присваивания ИЛИ)
- && (И)
  - о &&= (Оператор логического присваивания И)
- ! (НЕ)
- ?? (Оператор нулевого слияния)
  - о ??= (Оператор нулевого присваивания)

Несмотря на своё название, данные операторы могут применяться к значениям любых типов. Полученные результаты также могут иметь различный тип.

# || (или)

Традиционно в программировании ИЛИ предназначено только для манипулирования булевыми значениями: в случае, если какой-либо из аргументов true, он вернёт true, в противоположной ситуации возвращается false.

В JavaScript этот оператор работает несколько иным образом. Если значение не логического типа, то оно к нему приводится в целях вычислений.

Существует всего четыре возможные логические комбинации

```
alert( true || true ); // true
alert( false || true ); // true
alert( true || false ); // true
alert( false || false ); // false
```

# ИЛИ || находит первое истинное значение

Оператор || выполняет следующие действия:

- Вычисляет операнды слева направо.
- Каждый операнд конвертирует в логическое значение. Если результат true, останавливается и возвращает исходное значение операнда.
- Если все операнды являются ложными (false), возвращает последний из них.

Значение возвращается в исходном виде, без преобразования.

Другими словами, цепочка ИЛИ || возвращает первое истинное значение или последнее, если такое значение не найдено

```
alert( true || true ); // true
alert( false || true ); // true
alert( true || false ); // true
alert( false || false ); // false
alert( 1 || 0 ); // 1 (1 - истинное значение)
alert( true || 'какая-то строка' ); // true
alert( null || 1 ); // 1 (первое истинное значение)
alert( null || 0 || 1 ); // 1 (первое истинное значение)
alert( undefined || null || 0 ); // 0 (поскольку все ложно, возвращается последнее)
```



# ||= (Логическое присваивание ИЛИ)

Оператор `||=` принимает два операнда и выполняет следующие действия:

- Вычисляет операнды слева направо.
- Конвертирует `a` в логическое значение.
- Если `a` ложно, присваивает `a` значение `b`.

Концепция оператора `||=` заключается в «сокращённом вычислении»

```
let arturHasFunkoPop = false;

if (arturHasFunkoPop == false) {
  arturHasFunkoPop = "У Артура нет Funko Pop'a! :(";
}
alert(arturHasFunkoPop); // "У Артура нет Funko Pop'a! :("

let manufacturer = "";

if (manufacturer == false) {
  manufacturer = "Неизвестный производитель";
}
alert(manufacturer); // "Неизвестный производитель"
```

# && (И)

В традиционном программировании `И` возвращает `true`, если оба аргумента истинны, а иначе – `false`

```
alert( true && true );    // true
alert( false && true );   // false
alert( true && false );   // false
alert( false && false );  // false
```

```
let hour = 12;
let minute = 30;

if (hour == 12 && minute == 30) {
  alert( 'Время 12:30' );
}
```

## И && находит первое ложное значение

Оператор && выполняет следующие действия:

- Вычисляет операнды слева направо.
- Каждый операнд преобразует в логическое значение. Если результат false, останавливается и возвращает исходное значение этого операнда.
- Если все операнды были истинными, возвращается последний.

Другими словами, И возвращает первое ложное значение. Или последнее, если ничего не найдено.

```
// Если первый операнд истинный,  
// И возвращает второй:  
alert( 1 && 0 ); // 0  
alert( 1 && 5 ); // 5  
  
// Если первый операнд ложный,  
// И возвращает его. Второй операнд игнорируется  
alert( null && 5 ); // null  
alert( 0 && "какая-то строка" ); // 0
```

## &&= (Логическое присваивание И)

Принцип действия &&= практически такой же, как и у оператора логического присваивания ИЛИ ||=. Единственное отличие заключается в том, что &&= присвоит a значение b только в том случае, если a истинно.

Концепция оператора логического присваивания И &&= также основывается на «сокращённом вычислении»

```
let greeting = "Привет"; // строка непустая,  
// поэтому будет преобразована к логическому  
// значению true оператором &&=  
  
greeting &&= greeting + ", пользователь!";  
// то же самое, что true && (greeting = greeting + "...")  
  
alert(greeting) // "Привет, пользователь!"
```

# ! (НЕ)

```
alert( !true ); // false  
alert( !0 ); // true
```

Оператор принимает один аргумент и выполняет следующие действия:

1. Сначала приводит аргумент к логическому типу true/false.
2. Затем возвращает противоположное значение.

В частности, двойное НЕ !! используют для преобразования значений к логическому типу

Приоритет НЕ ! является наивысшим из всех логических операторов, поэтому он всегда выполняется первым, перед && или ||.

# Оператор нулевого слияния (??)

Оператор нулевого слияния представляет собой два вопросительных знака ??.

Так как он обрабатывает null и undefined одинаковым образом, то для этой темы мы введём специальный термин. Для краткости будем говорить, что значение «определено», если оно не равняется ни null, ни undefined.

Результат выражения a ?? b будет следующим:

- если a определено, то a,
- если a не определено, то b.

Иначе говоря, оператор ?? возвращает первый аргумент, если он не null/undefined, иначе второй.

```
let user;  
  
alert(user ?? "Аноним"); // Аноним (user не существует)
```

## Итого

- Оператор нулевого слияния `??` – это быстрый способ выбрать первое «определённое» значение из списка. Используется для присвоения переменным значений по умолчанию.
- Оператор `??` имеет очень низкий приоритет, лишь немного выше, чем у `?` и `=`, поэтому при использовании его в выражении, скорее всего, потребуются скобки.
- Запрещено использовать вместе с `||` или `&&` без явно указанного приоритета, то есть без скобок.
- Для присвоения переменной значения в зависимости от того, «определена» она или нет, используется оператор нулевого присваивания `??=`.



10

## ЦИКЛЫ WHILE И FOR

# Циклы while и for

## Цикл while

Цикл while имеет следующий синтаксис

Код из тела цикла выполняется, пока условие condition истинно.

Одно выполнение тела цикла по-научному называется итерация. Цикл в примере выше совершает три итерации.

Если бы строка `i++` отсутствовала в примере выше, то цикл бы повторялся (в теории) вечно. На практике, конечно, браузер не позволит такому случиться, он предоставит пользователю возможность остановить «подвисший» скрипт, а JavaScript на стороне сервера придётся «убить» процесс.

Любое выражение или переменная может быть условием цикла, а не только сравнение: условие while вычисляется и преобразуется в логическое значение.

Например, `while (i)` – более краткий вариант `while (i != 0)`.

```
while (condition) {  
    // код  
    // также называемый "телом цикла"  
}
```

```
let i = 3;  
  
while (i) { // когда i будет = 0,  
    // условие станет ложным, и цикл остановится  
  
    alert( i );  
    i--;  
}
```

# Цикл do...while

Проверку условия можно разместить под телом цикла, используя специальный синтаксис do..while

```
do {  
    // тело цикла  
} while (condition);
```

Цикл сначала выполнит тело, а затем проверит условие condition, и пока его значение равно true, он будет выполняться снова и снова.

Такая форма синтаксиса оправдана, если вы хотите, чтобы тело цикла выполнилось хотя бы один раз, даже если условие окажется ложным. На практике чаще используется форма с предусловием: while(...) {...}.

```
let i = 0;  
do {  
    alert( i );  
    i++;  
} while (i < 3);
```

# Циклы while и for

## Цикл for

```
for (начало; условие; шаг) {  
    // ... тело цикла ...  
}
```

Более сложный, но при этом самый распространённый цикл — цикл for.

Давайте разберёмся, что означает каждая часть, на примере.

Цикл примера выполняет alert(i) для i от 0 до (но не включая) 3

```
for (let i = 0; i < 3; i++) { // выведет 0, затем 1, затем 2  
    alert(i);  
}
```

Рассмотрим конструкцию for подробнее

| Часть   | Код       | Описание                                                                 |
|---------|-----------|--------------------------------------------------------------------------|
| начало  | let i = 0 | Выполняется один раз при входе в цикл                                    |
| условие | i < 3     | Проверяется перед каждой итерацией цикла.                                |
| тело    | alert(i)  | Выполняется снова и снова, пока условие вычисляется в true.              |
| шаг     | i++       | Выполняется после тела цикла на каждой итерации перед проверкой условия. |



## Циклы while и for

### Пропуск частей for

Любая часть for может быть пропущена.

Для примера, мы можем пропустить начало если нам ничего не нужно делать перед стартом цикла.

```
let i = 0; // мы уже имеем объявленную i с присвоенным значением

for (; i < 3; i++) { // нет необходимости в "начале"
  alert( i ); // 0, 1, 2
}
```

Можно убрать и шаг, это сделает цикл аналогичным while (i < 3).

```
let i = 0;

for (; i < 3;) {
  alert( i++ );
}
```

А можно и вообще убрать всё, получив бесконечный цикл. При этом сами точки с запятой ; обязательно должны присутствовать, иначе будет ошибка синтаксиса.

```
for (;;) {
  // будет выполняться вечно
}
```

# Прерывание цикла: break

Обычно цикл завершается при вычислении условия в false.

Но мы можем выйти из цикла в любой момент с помощью специальной директивы break.

```
let sum = 0;

while (true) {

    let value = +prompt("Введите число", '');

    if (!value) break; // (*)

    sum += value;

}

alert( 'Сумма: ' + sum );
```

Директива break в строке (\*) полностью прекращает выполнение цикла и передаёт управление на строку за его телом, то есть на alert.

Вообще, сочетание «бесконечный цикл + break» – отличная штука для тех ситуаций, когда условие, по которому нужно прерваться, находится не в начале или конце цикла, а посередине или даже в нескольких местах его тела.

# Переход к следующей итерации: continue

Директива `continue` – «облегчённая версия» `break`. При её выполнении цикл не прерывается, а переходит к следующей итерации (если условие все ещё равно `true`). Её используют, если понятно, что на текущем повторе цикла делать больше нечего.

Для чётных значений `i`, директива `continue` прекращает выполнение тела цикла и передаёт управление на следующую итерацию `for` (со следующим числом). Таким образом `alert` вызывается только для нечётных значений.

```
for (let i = 0; i < 10; i++) {  
    // если true, пропустить оставшуюся часть тела цикла  
    if (i % 2 == 0) continue;  
  
    alert(i); // 1, затем 3, 5, 7, 9  
}
```

# Метки для break/continue

Бывает, нужно выйти одновременно из нескольких уровней цикла сразу.

Например, в коде мы проходимся циклами по *i* и *j*, запрашивая с помощью `prompt` координаты (*i*, *j*) с (0,0) до (2,2):

```
for (let i = 0; i < 3; i++) {  
  for (let j = 0; j < 3; j++) {  
    let input = prompt(`Значение на координатах (${i},${j})`, '');  
    // Что если мы захотим перейти к Готово (ниже) прямо отсюда?  
  }  
}  
  
alert('Готово!');
```

Нам нужен способ остановить выполнение, если пользователь отменит ввод.

Обычный `break` после `input` лишь прервёт внутренний цикл, но этого недостаточно.

Достичь желаемого поведения можно с помощью меток.

# Метки для break/continue

Метка имеет вид идентификатора с двоеточием перед циклом

```
labelName: for (...) {  
    ...  
}
```

```
outer: for (let i = 0; i < 3; i++) {  
    for (let j = 0; j < 3; j++) {  
        let input = prompt(`Значение на координатах (${i},${j})`, '');  
  
        // если пустая строка или Отмена, то выйти из обоих циклов  
        if (!input) break outer; // (*)  
  
        // сделать что-нибудь со значениями...  
    }  
}  
  
alert('Готово!');
```

Вызов `break <labelName>` в цикле ниже ищет ближайший внешний цикл с такой меткой и переходит в его конец. В примере это означает, что вызовом `break outer` будет разорван внешний цикл до метки с именем `outer`.

Таким образом управление перейдет со строки, помеченной (\*), к `alert('Готово!')`.

# Конструкция switch

Конструкция switch заменяет собой сразу несколько if.

Она представляет собой более наглядный способ сравнить выражение сразу с несколькими вариантами. Конструкция switch имеет один или более блок case и необязательный блок default.

Переменная x проверяется на строгое равенство первому значению value1, затем второму value2 и так далее.

Если соответствие установлено – switch начинает выполняться от соответствующей директивы case и далее, до ближайшего break (или до конца switch).

Если ни один case не совпал – выполняется (если есть) вариант default.

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Маловато' );
    break;

  // case 4:
  //   alert( 'В точку!' );
  //   break;

  case 5:
    alert( 'Перебор' );
    break;
  default:
    alert( "Нет таких значений" );
}
```

# Конструкция switch

Здесь оператор switch последовательно сравнит а со всеми вариантами из case.

Сначала 3, затем – так как нет совпадения – 4. Совпадение найдено, будет выполнен этот вариант, со строки alert( 'В точку!' ) и далее, до ближайшего break, который прервёт выполнение.

Если break нет, то выполнение пойдёт ниже по следующим case, при этом остальные проверки игнорируются.

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Маловато' );
    break;

  // case 4:
  //   alert( 'В точку!' );
  //   break;

  case 5:
    alert( 'Перебор' );
    break;
  default:
    alert( "Нет таких значений" );
}
```

# Группировка case

Несколько вариантов case, использующих один код, можно группировать.

Для примера, выполним один и тот же код для case 3 и case 5, сгруппировав их

Теперь оба варианта 3 и 5 выводят одно сообщение.

Возможность группировать case – это побочный эффект того, как switch/case работает без break. Здесь выполнение case 3 начинается со строки (\*) и продолжается в case 5, потому что отсутствует break.

```
let a = 3;

switch (a) {
  case 4:
    alert('Правильно!');
    break;

  case 3: // (*) группируем оба case
  case 5:
    alert('Неправильно!');
    alert("Может вам посетить урок математики?");
    break;

  default:
    alert('Результат выглядит странновато. Честно.');
```



# Тип имеет значение

Нужно отметить, что проверка на равенство всегда строгая. Значения должны быть одного типа, чтобы выполнялось равенство.

1. Для '0' и '1' выполнится первый alert.
2. Для '2' - второй alert.
3. Но для 3, результат выполнения prompt будет строка "3", которая не соответствует строгому равенству === с числом 3. Таким образом, мы имеем «мёртвый код» в case 3! Выполнится вариант default.

```
let arg = prompt("Введите число?");
switch (arg) {
  case '0':
  case '1':
    alert( 'Один или ноль' );
    break;

  case '2':
    alert( 'Два' );
    break;

  case 3:
    alert( 'Никогда не выполнится!' );
    break;
  default:
    alert( 'Неизвестное значение' );
}
```

11

# ФУНКЦИИ

## Объявление функции

```
function showMessage() {  
    alert( 'Всем привет!' );  
}
```

Для создания функций мы можем использовать объявление функции.

Вначале идёт ключевое слово `function`, после него имя функции, затем список параметров в круглых скобках через запятую (в вышеприведённом примере он пустой) и, наконец, код функции, также называемый «телом функции», внутри фигурных скобок

Вызов `showMessage()` выполняет код функции.

Здесь мы увидим сообщение дважды.

Этот пример явно демонстрирует одно из главных предназначений функций: избавление от дублирования кода.

Если понадобится поменять сообщение или способ его вывода – достаточно изменить его в одном месте: в функции, которая его выводит.

```
function showMessage() {  
    alert( 'Всем привет!' );  
}  
  
showMessage();  
showMessage();
```

## Локальные переменные

Переменные, объявленные внутри функции, видны только внутри этой функции.

```
function showMessage() {  
  let message = "Привет, я JavaScript!";  
  // локальная переменная  
  
  alert( message );  
}  
  
showMessage(); // Привет, я JavaScript!  
  
alert( message ); // <-- будет ошибка,  
// т.к. переменная видна только внутри функции
```

## Внешние переменные

У функции есть доступ к внешним переменным. Внешняя переменная используется, только если внутри функции нет такой локальной.

Если одноимённая переменная объявляется внутри функции, тогда она перекрывает внешнюю.

```
let userName = 'Artur';  
  
function showMessage() {  
  let message = 'Привет, ' + userName;  
  alert(message);  
}  
  
showMessage(); // Привет, Artur
```

# Глобальные переменные

Глобальные переменные видимы для любой функции (если только их не перекрывают одноимённые локальные переменные). Желательно сводить использование глобальных переменных к минимуму. В современном коде обычно мало или совсем нет глобальных переменных. Хотя они иногда полезны для хранения важнейших «общепроектных» данных

```
let userName = 'Rustam'; // Внешняя глобальная переменная

function showMessage() {
  let userName = "Artur"; // объявляем локальную переменную
  let message = 'Привет, ' + userName; // Artur
  alert(message);
}

// функция создаст и будет использовать свою собственную локальную переменную username
showMessage();

alert( userName ); // Artur, не изменилась, функция не трогала внешнюю переменную
```

## Параметры

Мы можем передать внутрь функции любую информацию, используя параметры.

```
function showMessage(from, text) { // параметры: from, text  
  alert(from + ': ' + text);  
}
```

```
showMessage('Аня', 'Привет!'); // Аня: Привет! (*)  
showMessage('Аня', "Как дела?"); // Аня: Как дела? (**)
```

Когда функция вызывается в строках (\*) и (\*\*), переданные значения копируются в локальные переменные `from` и `text`. Затем они используются в теле функции.

Значение, передаваемое в качестве параметра функции, также называется аргументом.

Другими словами:

- Параметр – это переменная, указанная в круглых скобках в объявлении функции.
- Аргумент – это значение, которое передаётся функции при её вызове.

Мы объявляем функции со списком параметров, затем вызываем их, передавая аргументы.

# Значения по умолчанию

Если при вызове функции аргумент не был указан, то его значением становится `undefined`.

Например, функция `showMessage(from, text)`, что рассматривалась ранее может быть вызвана с одним аргументом.

```
showMessage("Аня");
```

Это не приведёт к ошибке. Такой вызов выведет `"*Аня*: undefined"`.

В вызове не указан параметр `text`, поэтому предполагается, что `text === undefined`.

Если мы хотим задать параметру `text` значение по умолчанию, мы должны указать его после =

```
function showMessage(from, text = "текст не добавлен") {  
  | alert( from + ": " + text );  
}
```

```
showMessage("Аня"); // Аня: текст не добавлен
```

## Возврат значения

```
function sum(a, b) {  
  return a + b;  
}
```

```
let result = sum(1, 2);  
alert( result ); // 3
```

Функция может вернуть результат, который будет передан в вызвавший её код.

Простейшим примером может служить функция сложения двух чисел.

Директива `return` может находиться в любом месте тела функции. Как только выполнение доходит до этого места, функция останавливается, и значение возвращается в вызвавший её код (присваивается переменной `result` выше). Вызовов `return` может быть несколько.

Возможно использовать `return` и без значения.

Это приведёт к немедленному выходу из функции

```
function checkAge(age) {  
  if (age >= 18) {  
    return true;  
  } else {  
    return confirm('А родители разрешили?');  
  }  
}  
  
let age = prompt('Сколько вам лет?', 18);  
  
if ( checkAge(age) ) {  
  alert( 'Доступ получен' );  
} else {  
  alert( 'Доступ закрыт' );  
}
```



## Выбор имени функции

Функция – это действие. Поэтому имя функции обычно является глаголом. Оно должно быть кратким, точным и описывать действие функции, чтобы программист, который будет читать код, получил верное представление о том, что делает функция.

Как правило, используются глагольные префиксы, обозначающие общий характер действия, после которых следует уточнение. Обычно в командах разработчиков действуют соглашения, касающиеся значений этих префиксов.

Например, функции, начинающиеся с "show" обычно что-то показывают.

Функции, начинающиеся с:

- "get..." - возвращают значение,
- "calc..." - что-то вычисляют,
- "create..." - что-то создают,
- "check..." - что-то проверяют и возвращают логическое значение, и т.д.

```
showMessage()    // показывает сообщение
getAge()         // возвращает возраст (получая его каким-то образом)
calcSum()        // вычисляет сумму и возвращает результат
createForm()     // создаёт форму (и обычно возвращает её)
checkPermission() // проверяет доступ, возвращая true/false
```

# Выбор имени функции

Благодаря префиксам становится понятным, что делает её код, и какое значение она может возвращать. Функция должна делать только то, что явно подразумевается её названием. И это должно быть одним действием.

Два независимых действия обычно подразумевают две функции, даже если предполагается, что они будут вызываться вместе (в этом случае можем создать третью функцию, вызывающую их).

Несколько примеров, которые нарушают это правило:

- `getAge` – будет плохим выбором, если функция будет выводить `alert` с возрастом (должна только возвращать его).
- `createForm` – будет плохим выбором, если функция будет изменять документ, добавляя форму в него (должна только создавать форму и возвращать её).
- `checkPermission` – плохой выбор, если функция будет отображать сообщение с текстом доступ разрешён/запрещён (должна только выполнять проверку и возвращать её результат).

**СПАСИБО ЗА ВНИМАНИЕ!**

12 февраля 2026 года